



HASHMAP INTERNAL IMPLEMENTATION!

"DO YOU KNOW HOW IT WORKS
INTERNALLY?"



@JavaWithKiran

What is HashMap?

- Part of Java Collections Framework
- Stores data as key-value pairs
- Allows one null key & multiple null values
- Not synchronized (not thread-safe)

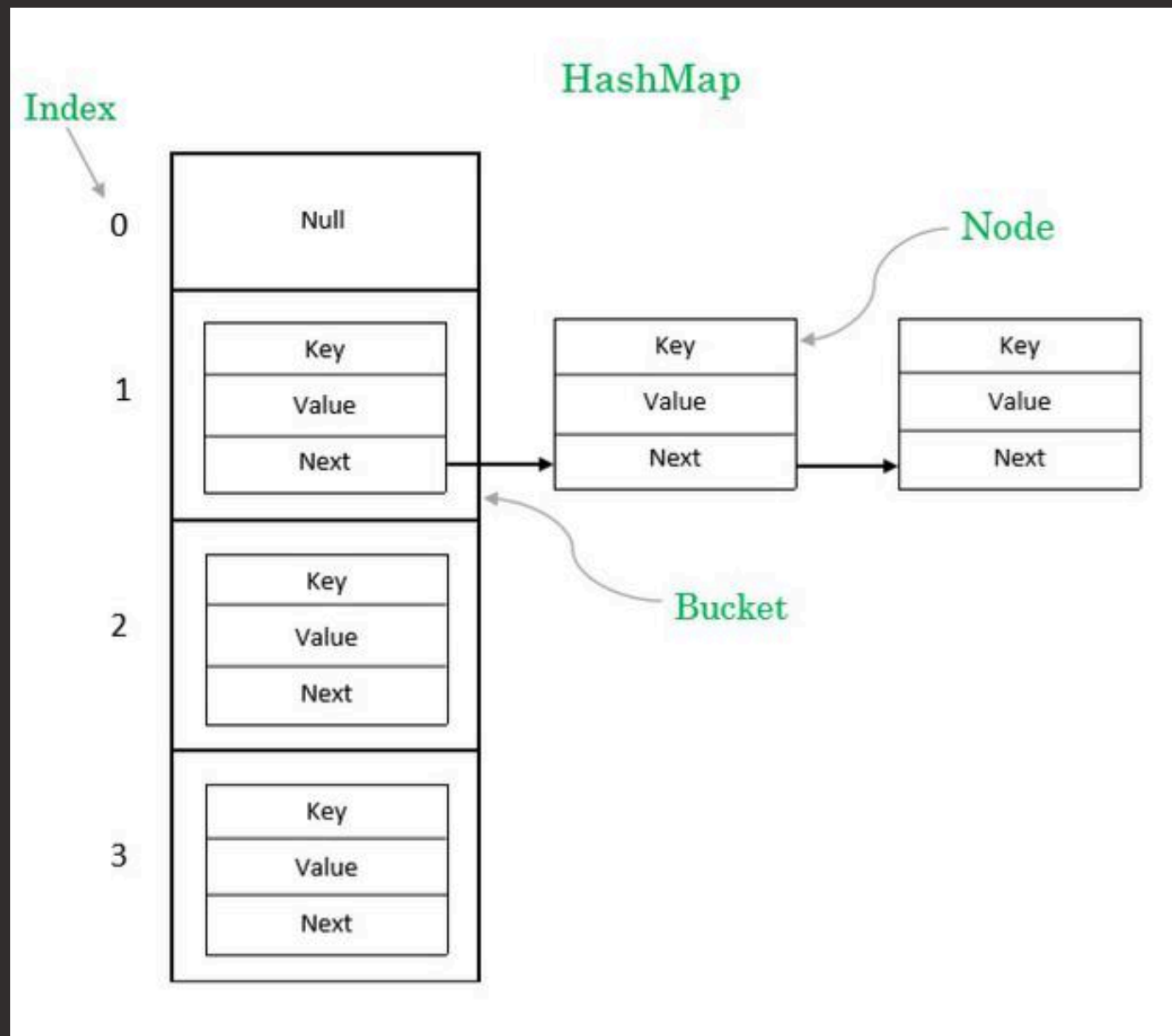
HashMap internally uses:

- Array (Bucket array)
- Node (LinkedList / TreeNode)

The Array [The Buckets]:

- The HashMap initializes an array.
- Each slot in this array is called a "bucket."

The Node: Each entry in the bucket is a Node that contains the **key**, the **value**, the **hashcode**, and a pointer to the next node.

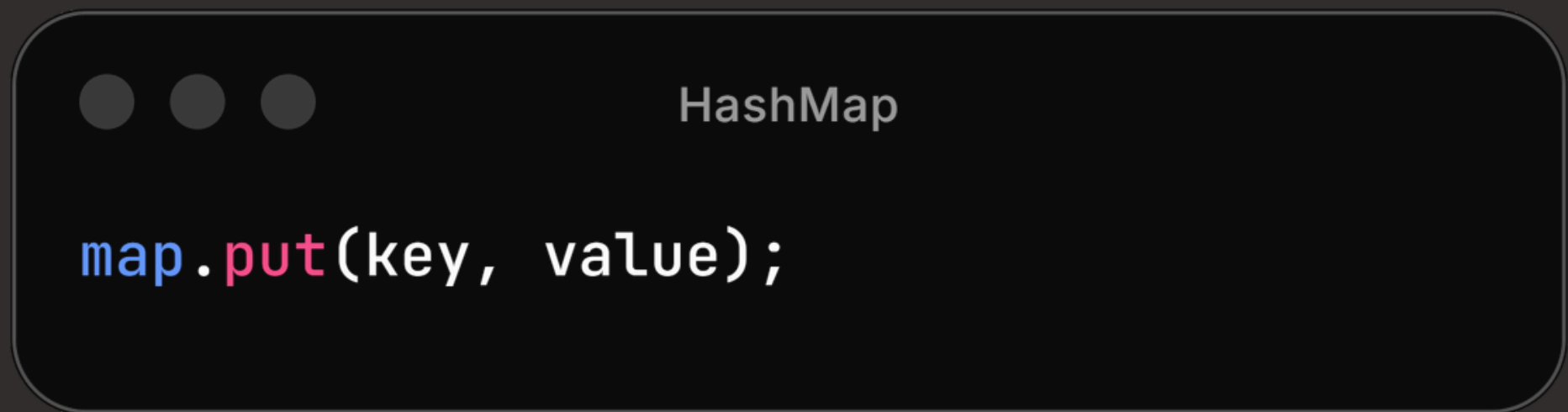


Structure = Array + LinkedList (or Tree)

Each bucket stores:

- Key
- Value
- Hash
- Next reference

How put() Works in Map?



Steps:

- Calculate hashCode() of key
- Convert hash → index
- Store in bucket

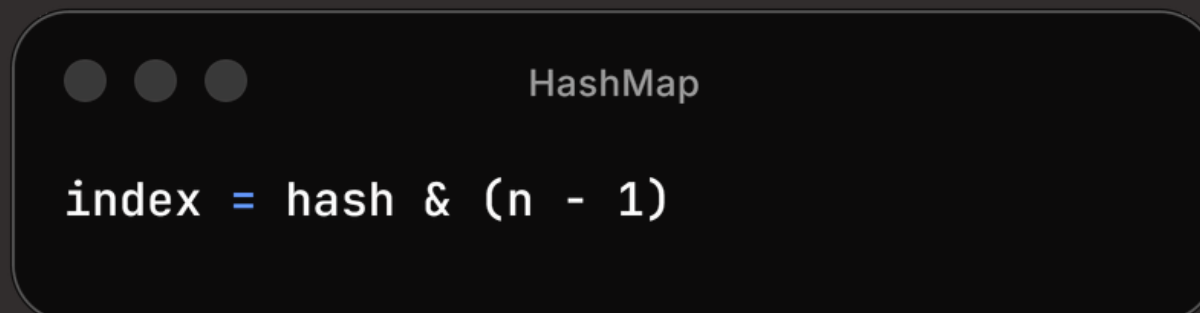
If bucket empty → Insert

If not → Handle collision

- put uses equals() to determine if a key already exists in a bucket so it can decide whether to update the existing value or add a new entry.

Hash Calculation

Index calculation:



```
index = hash & (n - 1)
```

Where:

- n = size of array (power of 2)

Faster than modulo (%)

Ensures uniform distribution

Collision Handling

If multiple keys land in same bucket:

Java 7 → LinkedList

Java 8+ →

- LinkedList → Tree (Red-Black Tree)

Condition:

- If bucket size ≥ 8 → Convert to Tree
- If ≤ 6 → Back to LinkedList

Improves performance to $O(\log n)$

Resize / Rehashing

When capacity exceeds threshold:

```
HashMap  
  
threshold = capacity × load factor  
(default load factor = 0.75)
```

- HashMap resizes (doubles size)

Process:

- Creates new array
- Recalculates index
- Moves elements

Called as **Rehashing**.

Time Complexity

```
HashMap  
  
Operation | Complexity  
----- | -----  
get()     | O(1) avg  
put()     | O(1) avg  
Worst    | O(log n)
```

ThankYou!

@JavaWithKiran